

Arquitectura de Software y Diseño de Flujo Lógico

Juego Síncrono "Piedra, Papel o Tijera" basado en el Patrón MVC

Autor: David Rafael Dueñas Ripalda

Docente Tutor: Mgs. Estefanía Vanessa Heredia Jiménez

Asignatura: LOGICA DE PROGRAMACION 1-SIN-1B

REINVENTEMOS
EL FUTURO

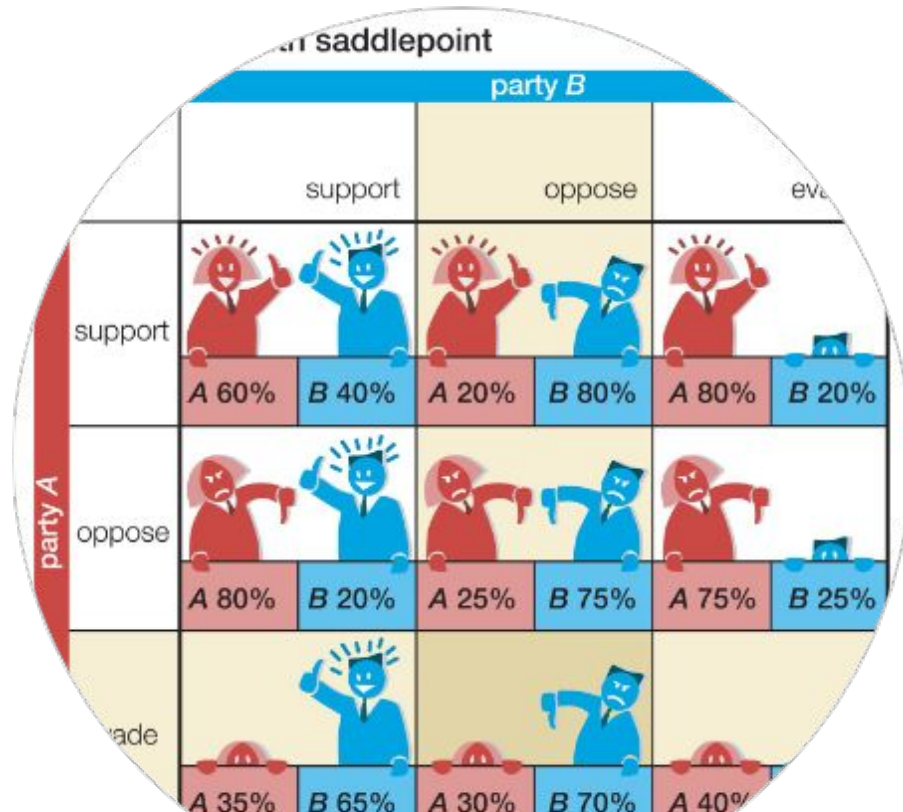
SECCIÓN 01

Fundamentación Científica

Análisis del modelado funcional y la Teoría de Juegos aplicada a la toma de decisiones del computador.

REINVENTEMOS
EL FUTURO

Teoría de Juegos y Equilibrio de Nash



Estrategia Mixta Óptima

Para garantizar una simulación justa, el motor de juego implementa una distribución de probabilidad uniforme (Equilibrio de Nash).

$$P(\text{Piedra}) = P(\text{Papel}) = P(\text{Tijera}) = \frac{1}{3}$$

Esto asegura que la CPU sea impredecible y el juego mantenga un estado de suma cero equitativo.

REINVENTEMOS
EL FUTURO

| Especificación de Requerimientos (RF)

-  RF-01 Gestión de Identidad: Registro del nombre del jugador para personalización de sesión.
-  RF-02 Captura de Entrada: Selección de jugada mediante menú validado (1-5).
-  RF-03 Lógica de CPU: Generador probabilístico uniforme para simular un oponente humano.
-  RF-04 Evaluación de Reglas: Contraste síncrono de jugadas concurrentes (Victoria/Derrota).
-  RF-05 Gestión de Marcador: Persistencia del estado acumulado durante la ejecución.
-  RF-06 Inicialización de Estado: Función explícita para resetear el historial de partidas.

REINVENTEMOS
EL FUTURO

| Estándares de Rendimiento (RNF)

100

Límite de Latencia (ms)

Eficiencia Temporal

El sistema debe procesar la lógica de negocio y actualizar la interfaz en un tiempo inferior a los 100ms para cumplir con el RNF-02.

Arquitectura Desacoplada

Implementación de un acoplamiento débil (RNF-01) mediante el patrón MVC, separando la lógica del motor de la interfaz de consola.

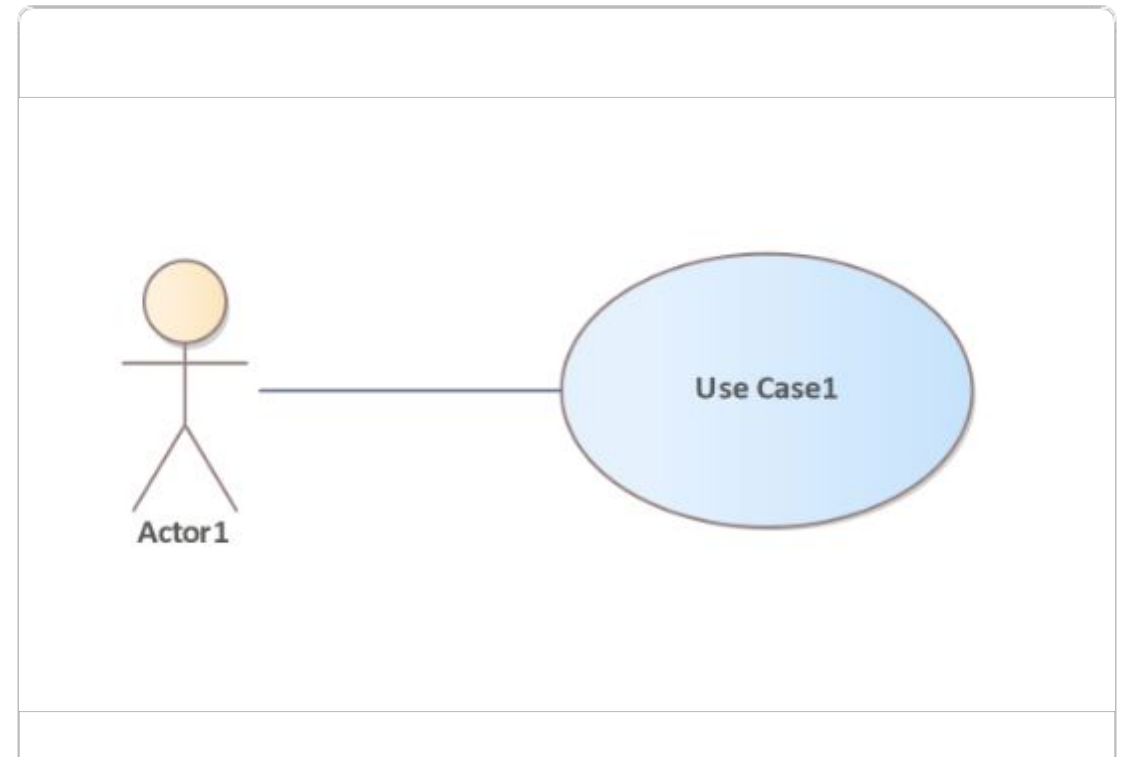
REINVENTEMOS
EL FUTURO

Modelado de Casos de Uso UML

Interacciones del Jugador

El diagrama de casos de uso define los límites del sistema y las acciones permitidas al actor primario:

- **Inicio:** Registro de identificador textual.
- **Turno:** Selección de jugada válida.
- **Mantenimiento:** Reinicio de marcador general.
- **Cierre:** Terminación controlada del proceso.

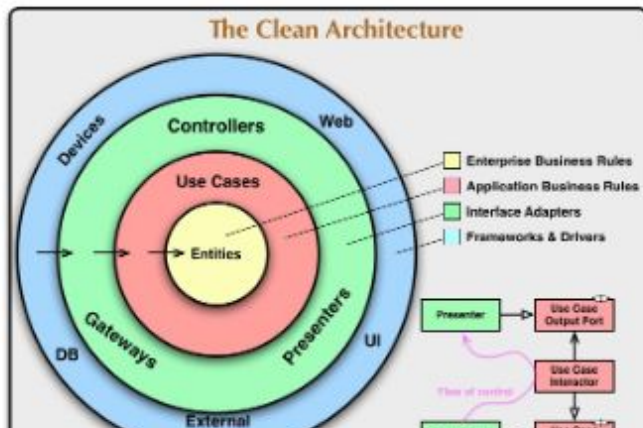


REINVENTEMOS
EL FUTURO

Arquitectura Modular por Capas

Patrón de Diseño MVC

La organización del código fuente se divide en capas jerárquicas para facilitar el mantenimiento y la escalabilidad tecnológica.



Interacción Síncrona: El controlador orquesta las llamadas entre la entrada del usuario y las reglas matemáticas.

Independencia de Vista: La lógica de juego no conoce la implementación de la consola, permitiendo futuras interfaces web.

REINVENTEMOS
EL FUTURO

Capa de Dominio: El Modelo



GameEngine

Contiene el motor de reglas y la resolución de colisiones de jugadas.



Scoreboard

Gestiona la persistencia temporal de victorias, derrotas y empates.



Player Class

Representa al sujeto de pruebas y sus atributos de sesión.

REINVENTEMOS
EL FUTURO

| Capa de Presentación: La Vista



ConsoleInterface

Traduce las entradas de teclado en comandos del dominio.



Output Formatter

Despliega el marcador general y los resultados detallados por ronda.



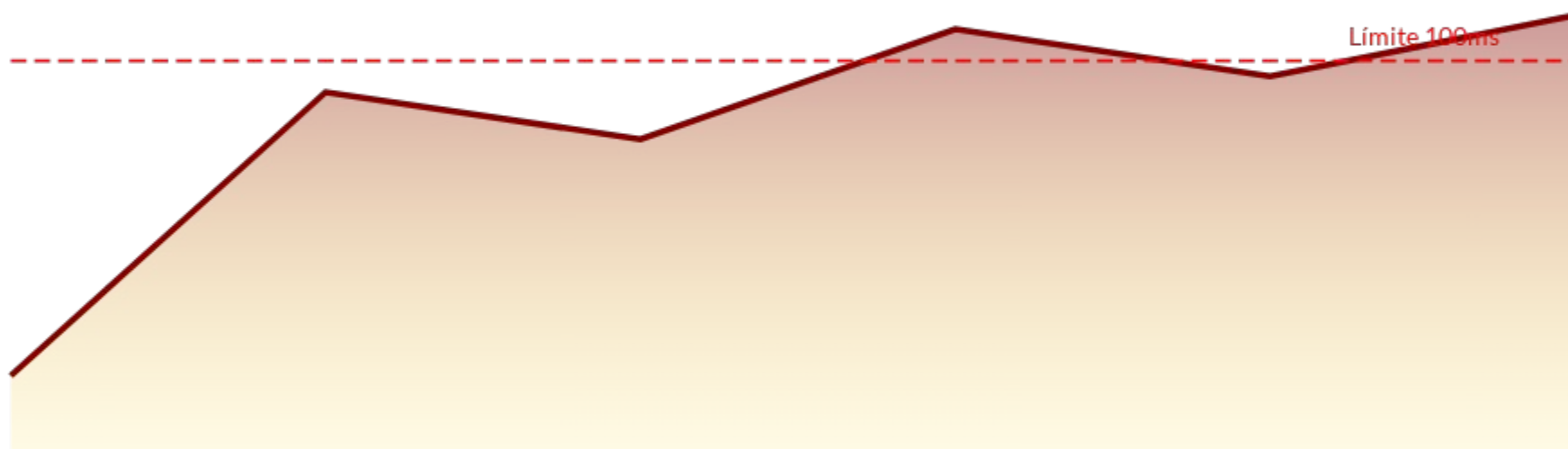
Notifications

Emite alertas sobre el estado del sistema (Reinicio, Errores).

REINVENTEMOS
EL FUTURO

Eficiencia Temporal del Sistema

Simulación de latencia de procesamiento por ronda (Objetivo < 100ms)



REINVENTEMOS
EL FUTURO

Transferencia y Escalabilidad

Preparado para el Futuro

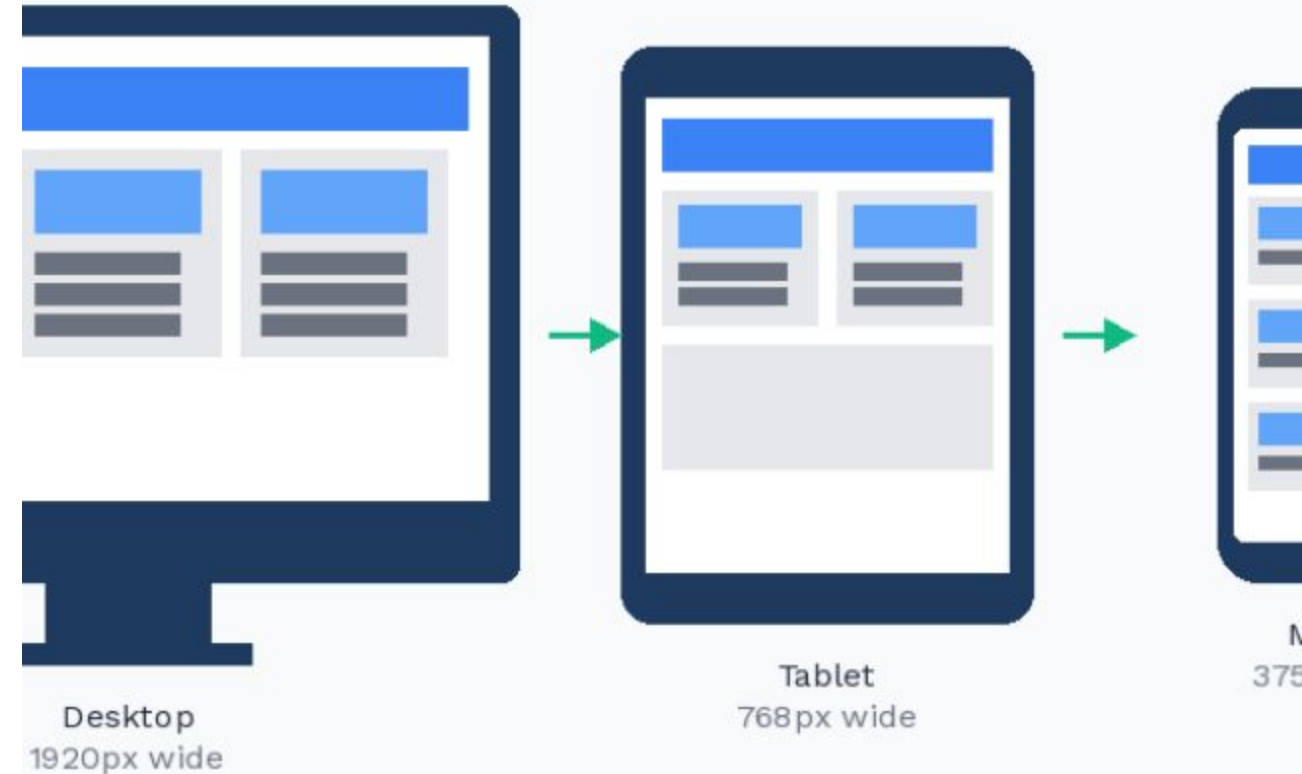
Gracias al diseño modular MVC, el sistema permite migraciones tecnológicas fluidas:

- Migración de Consola a Aplicación Web.
- Integración con bases de datos para persistencia real.
- Despliegue como API de servicios.

El "Core" matemático permanece intacto ante cambios en la interfaz.

Responsive Web Design

One website that adapts to every screen



REINVENTEMOS
EL FUTURO



GRACIAS POR SU ATENCION

LOGICA DE PROGRAMACIÓN | David Rafael Dueñas Ripalda

REINVENTEMOS
EL FUTURO